Mini Assessment

Name	M. Mirudhula Daisy
Reg No	9923005207
Department	ECE

1. If an API returns 429, what should your program do?

Answer: HTTP 429 tells the program that it has exceeded the number of allowed requests in a given time window. Rather than retrying right away, the program should pause execution, ideally for the duration given in the Retry-After header. If that header isn't available, it should use a backoff strategy where the wait time increases with each retry, and it should cap the total number of retries so the program doesn't get stuck. If every retry still fails, the program should fail gracefully and report the issue instead of freezing or crashing.

2. If response.json() fails, what should your program do?

Answer: response.json() fails when the returned content isn't valid JSON, for example if the API sends back an error page, an empty response, or a partially downloaded body. The safest approach is to place the call inside a try-except block, catch the JSON decoding error specifically, and fall back to inspecting response.status_code and response.text to see what was actually returned. The failure should be logged so the rest of the program can continue processing other records instead of stopping entirely.

3. If a JSON key is missing, why might .get() be safer than []?

Answer: .get() is safer than [] because missing keys are common in real JSON data — not every record is guaranteed to have every field. data['key'] immediately raises a KeyError and halts the program if the key is absent, whereas data.get('key', default) simply returns a default value (like 'N/A' or 'Pending') and lets execution continue smoothly. This is especially important when processing many records, since one missing field shouldn't be able to crash the entire report.